ResNet-101 Training Configuration

Official repository and PyTorch reference-recipe audit • TBX11K multi-label classification • 512 × 512 • from scratch • 70-epoch maximum • RTX 4050 Laptop GPU (6 GB)

1. Official source and scope

Primary original repository: KaimingHe/deep-residual-networks. It identifies the original ResNet-50/101/152 models and reports ImageNet results, but says the supplied Caffe models were converted from the authors' implementation and were not trained using that later Caffe version. Its README does not publish a complete training command with every optimizer, batch, schedule, and augmentation setting. This document separates original-repository facts, the official Torchvision reference recipe, and project-specific adaptations.

Important: "official ResNet hyperparameters" is not one universal recipe. The original Caffe repository, original paper, and current Torchvision reference training code are distinct sources. The proposed settings are not claimed to reproduce the original ImageNet experiment exactly.

2. Parameter-by-parameter comparison

Parameter	Original source / official PyTorch reference	Use in this experiment	Why / status
Architecture	Original ResNet-101; Torchvision bottleneck counts [3,4,23,3].	Torchvision ResNet-101; final FC becomes 2 logits.	Torchvision uses ResNet V1.5 stride placement, differing from original paper.
Parameters / compute	Torchvision: 44,549,160 parameters; about 7.80 GFLOPs at 224×224.	Measure actual memory and compute at 512×512.	Do not present 224-input FLOPs as the 512-input cost.
Input	Original ImageNet evaluation: 224×224 crop after resize to shorter side 256.	512×512 RGB, project requirement.	Project adaptation, not official ImageNet preprocessing.
Initialization	Original repository supplies converted models for testing/fine-tuning; notes they were not trained using the provided Caffe version.	`resnet101(weights=None)`; random initialization.	Matches from-scratch requirement; do not load DEFAULT weights.
Task / labels	ImageNet-1K, 1,000-way single-label classification.	Two independent outputs: Active TB, Obsolete Pulmonary TB.	All combinations allowed: [0,0], [1,0], [0,1], [1,1].
Output / loss	ImageNet recipe uses class target and cross-entropy.	2 logits + `BCEWithLogitsLoss` on float target vector.	No softmax/four-class target; do not sigmoid before BCEWithLogitsLoss.
Batch / GPUs	Torchvision reference: batch size 32 per GPU; generally 8×V100 runs.	Start batch 4; fallback 2 if OOM; one RTX 4050 6 GB.	Hardware adaptation; official multi-GPU batch/LR not portable directly.
Epochs	Torchvision reference recipe: 90 epochs.	70 epochs maximum; early-stopping patience 20.	Project budget and stopping policy; not official and not proof of convergence.
Optimizer / LR	Torchvision: SGD, LR 0.1, momentum 0.9, weight decay 1e-4.	Start SGD LR 0.01, momentum 0.9, weight decay 1e-4; Nesterov off.	LR 0.01 is a conservative adaptation, not an official value.

ResNet-101 Training Configuration — continued

3. Remaining hyperparameters and implementation choices

Parameter	Official source	Proposed project value	Notes
LR scheduler	Torchvision ResNet reference: StepLR, step size 30, gamma 0.1.	StepLR(step_size=30, gamma=0.1); one decay at epoch 30.	Do not add epoch-40 drop unless separately tested.
Momentum / decay	Momentum 0.9; weight decay 1e-4.	Keep both unchanged.	Reference values.
Data workers	16 workers in reference script defaults.	Start with 2 on Windows; try 4 if stable.	Practical adaptation, not official.
Precision / memory	Reference script supports AMP as an option; original repo does not specify PyTorch AMP.	CUDA FP16 autocast + GradScaler; small batch.	Test a forward/backward pass before full run.
Normalization	Torchvision ImageNet transform: mean [0.485,0.456,0.406], std [0.229,0.224,0.225], 224 crop.	Reuse DenseNet baseline preprocessing: RGB, 512×512, same normalization.	Controlled comparison choice, not a claim of optimal X-ray normalization.
Augmentation	Original README lacks a complete portable transform recipe.	Same policy as DenseNet baseline: no augmentation initially.	Test clinically defensible train-only augmentation separately.
Checkpoint	ImageNet benchmark reports errors; no TBX11K checkpoint rule.	Save best validation macro-F1 and last checkpoint.	Never select using test results.
Seed / split	Original ImageNet setup differs from project split.	Seed 42; reuse fixed manifests.	No new random split.
Evaluation	ImageNet top-1/top-5 error.	Per-label precision/recall/F1/specificity, ROC-AUC, PR-AUC, exact-match, Hamming loss, four-category report.	Same definitions; threshold 0.5 for direct baseline. Tune only on validation.

Reference recipe provenance: Official Torchvision classification documentation describes generally 8×V100 reference runs with batch size 32 per GPU, 90 epochs, LR 0.1, momentum 0.9, weight decay 1e-4, and StepLR decay every 30 epochs by 0.1. These are reference-recipe facts, not settings explicitly published in the original KaimingHe README.

ResNet-101 Training Configuration — implementation plan

4. Recommended configuration to implement

This is a project-adapted starting point informed by the official Torchvision reference recipe, not a verbatim reproduction of ImageNet training.

Setting	Recommended value	Status / rationale
Model	`torchvision.models.resnet101(weights=None)`; final `fc = nn.Linear(2048, 2)`.	Scratch model and task-specific output head.
Trainable weights	All parameters trainable; no pretrained checkpoint.	Required for from-scratch baseline.
Input / targets	512×512, 3 channels; targets `[active_tb, obsolete_tb]` in $\{0,1\}^2$.	Both = [1,1]; convert grayscale to RGB consistently.
Epochs / early stopping	70 epochs maximum; early-stopping patience 20.	Matches existing DenseNet-121 policy; 70 is a maximum, not a guarantee of convergence.
Batch	4; fallback 2 if CUDA OOM.	Test one forward/backward pass first.
Gradient accumulation	Default 1; optional 4 steps if needed.	Effective batch 16 at physical batch 4; optional.
Optimizer	`SGD(lr=0.01, momentum=0.9, weight_decay=1e-4)`; Nesterov off initially.	LR adapted from official 0.1 due to smaller batch/task.
Scheduler / loss	`StepLR(step_size=30, gamma=0.1)`; `BCEWithLogitsLoss()` unweighted initially.	Class weighting should be a separate experiment.
AMP / loader	FP16 autocast + GradScaler; `num_workers=2`, `pin_memory=True`.	Memory-saving, Windows-friendly starting point.
Selection / test	Best validation macro-F1 + last checkpoint; test chosen model once.	Do not choose checkpoint or thresholds using test data.

5. Copyable configuration summary

```
MODEL=resnet101(weights=None)
NUM_OUTPUTS=2  # ActiveTB, ObsoleteTB
INPUT_SIZE=(512, 512); WEIGHTS=None
EPOCHS=70
EARLY_STOPPING_PATIENCE=20
BATCH_SIZE=4  # fallback 2 if CUDA OOM
OPTIMIZER=SGD(lr=0.01, momentum=0.9, weight_decay=1e-4, nesterov=False)
SCHEDULER=StepLR(step_size=30, gamma=0.1)
LOSS=BCEWithLogitsLoss()
AMP=CUDA_FP16_WITH_GRADSCALER
NUM_WORKERS=2; SEED=42
CHECKPOINT=best_validation_macro_f1.pth
PREPROCESSING=RGB_512x512_same_as_DenseNet_baseline
AUGMENTATION=none_for_initial_fair_comparison
```

ResNet-101 Training Configuration — verification checklist

6. Checks before training

• Architecture: Confirm `resnet101(weights=None)`, `fc.out_features == 2`, bottleneck counts [3,4,23,3], and Torchvision V1.5 stride placement.

• From scratch: No pretrained weights; every backbone parameter trainable.

• Label order: Output 0 = ActiveTuberculosis; output 1 = ObsoletePulmonaryTuberculosis across dataset, loss, inference, reports.

• Loss: Raw logits to BCEWithLogitsLoss; sigmoid only for probabilities and thresholding.

• Splits: Reuse canonical classification manifests: 4,599 images (train 3,219; validation 690; test 690). Do not confuse with separate 8,400-image detection split.

• Fair comparison: Keep resolution, normalization, augmentation, seed, labels, metric formulas, and threshold consistent with DenseNet-121.

• Memory: Test one forward/backward pass at 512×512; reduce batch before changing resolution.

• Metrics: Use validation for model/threshold selection. Both category has only two test samples.

• Reproducibility: Log versions, config, seed, manifest paths, best epoch, validation macro-F1, checkpoint path, and curves.

7. What can and cannot be called “official”

Original ResNet source: KaimingHe’s repository is the primary source for the authors’ models and ImageNet results. It warns the supplied models were converted to a later Caffe version and were not trained using that version. It does not provide a complete portable hyperparameter recipe. It reports ResNet-101 single-crop ImageNet top-1/top-5 error of 23.6% / 7.1% and ten-crop error of 21.8% / 6.1%; these are historical ImageNet results, not expected TBX11K results.

Official PyTorch reference recipe: Torchvision documents 90 epochs, batch size 32 per GPU, SGD LR 0.1, momentum 0.9, weight decay 1e-4, StepLR step size 30 / gamma 0.1 for generally 8×V100 reference runs. Proposed batch, LR, input size, multi-label loss, and 70-epoch budget are adaptations. Torchvision ResNet-101 has 44.55M parameters and is the ResNet V1.5 variant.

Research wording: Call this “ResNet-101 trained from scratch using a project-adapted configuration informed by the official Torchvision reference recipe,” not an exact reproduction of official ImageNet training.

8. Sources checked

1. Kaiming He et al., original repository: github.com/KaimingHe/deep-residual-networks
2. Official Torchvision ResNet-101 docs: ResNet-101 model documentation
3. Official Torchvision reference README: classification README
4. Official Torchvision training script: classification/train.py
5. He, Zhang, Ren & Sun, “Deep Residual Learning for Image Recognition,” arXiv:1512.03385

Verification note: The original repository, Torchvision model documentation, reference README, and reference training script were cross-checked. Official-source facts and project adaptations are separated.